

تقرير مشروع البرمجة المتوازية

محرك المعالجة عالي الأداء لنظام التجارة الإلكترونية

High-Performance E-Commerce Backend Engine - Parallel Cart

البند	القيمة
اسماء الطلاب	طارق حسام الصباغ التاجي محمد معاذ محمد سامر سويد محمد احمد عاشور احمد طلال سعود محمد حمزة محمد رائد مقداد
المادة	البرمجة المتوازية
الفصل الدراسي	2026
التقنية الأساسية	Spring Boot + PostgreSQL + Redis + Kafka + Nginx
تاريخ التقرير	2026/6/20

فهرس التقرير

- 1. ملخص عام عن المشروع
- 2. فكرة المشروع وحدود العمل
- 3. المعمارية العامة للنظام
- 4. طريقة العمل على المتطلبات
- 5. تفصيل المتطلبات العشرة
- 6. عرض النتائج والقياسات

1. ملخص عام عن المشروع

تم بناء مشروع Parallel Cart كنظام خلفي لمتجر إلكتروني عالي الأداء. الهدف الأساسي من المشروع لم يكن زيادة عدد الصفحات أو الوظائف الشكلية، بل بناء مسار تجارة إلكترونية قادر على العمل تحت ضغط عال وتزامن كبير مع المحافظة على سلامة البيانات. لذلك تم التركيز على عمليات المنتجات، السلة، إتمام الطلب، الدفع، المخزون، والمعالجة الخلفية.

أخطر نقطة في المشروع هي عملية checkout لأنها تجمع أكثر من عملية حساسة في مسار واحد: قراءة السلة، حجز المخزون، إنشاء الطلب، تسجيل الدفع، حذف عناصر السلة، وإرسال حدث لإنشاء الفاتورة والتنبيه. لهذا السبب تم تصميم checkout كعملية محمية بعدة طبقات: backpressure، مفتاح idempotency، أقفال موزعة، optimistic locking، fallback باستخدام pessimistic locking، ومعاملة قاعدة بيانات واحدة.

المحور	ما تم تنفيذه
الوظائف الأساسية	منتجات، مستخدمون، سلة، عناصر سلة، طلبات، عناصر طلب، دفع، مخزون.
سلامة التزامن	Version على المخزون، retry مع jitter، قفل تشاؤمي عند الحاجة، وأقفال Redis/Redisson.
سلامة المعاملة	checkout كامل ضمن Transaction واحدة لضمان all-or-nothing.
المعالجة غير المتزامنة	Outbox داخل قاعدة البيانات ثم Kafka event ثم consumers للفواتير والتنبيهات.
الكاش	Redis cache للمنتجات والسلات والطلبات مع invalidation بعد نجاح المعاملة.
الضغط والقياس	JMeter/k6، SQL validation، Actuator metrics، bottleneck ranking.

2. فكرة المشروع وحدود العمل

فكرة المشروع هي محاكاة متجر إلكتروني يتعامل مع عدد كبير من الطلبات المتزامنة. الوظائف الأساسية موجودة فقط بالقدر الذي يسمح بتطبيق المتطلبات غير الوظيفية المطلوبة في المادة، مثل حماية المخزون من التضارب، التحكم بالموارد، استخدام batch jobs، queues، caching، load balancing، stress testing، و benchmarking. تم اعتماد Spring Boot لأنه مناسب لبناء backend منظم، ويعطي أدوات جاهزة لـ JPA، transactions، validation، AOP، Actuator، Kafka، Redis. كما تم الاعتماد على Docker Compose لتشغيل البيئة كاملة بشكل قابل للإعادة.

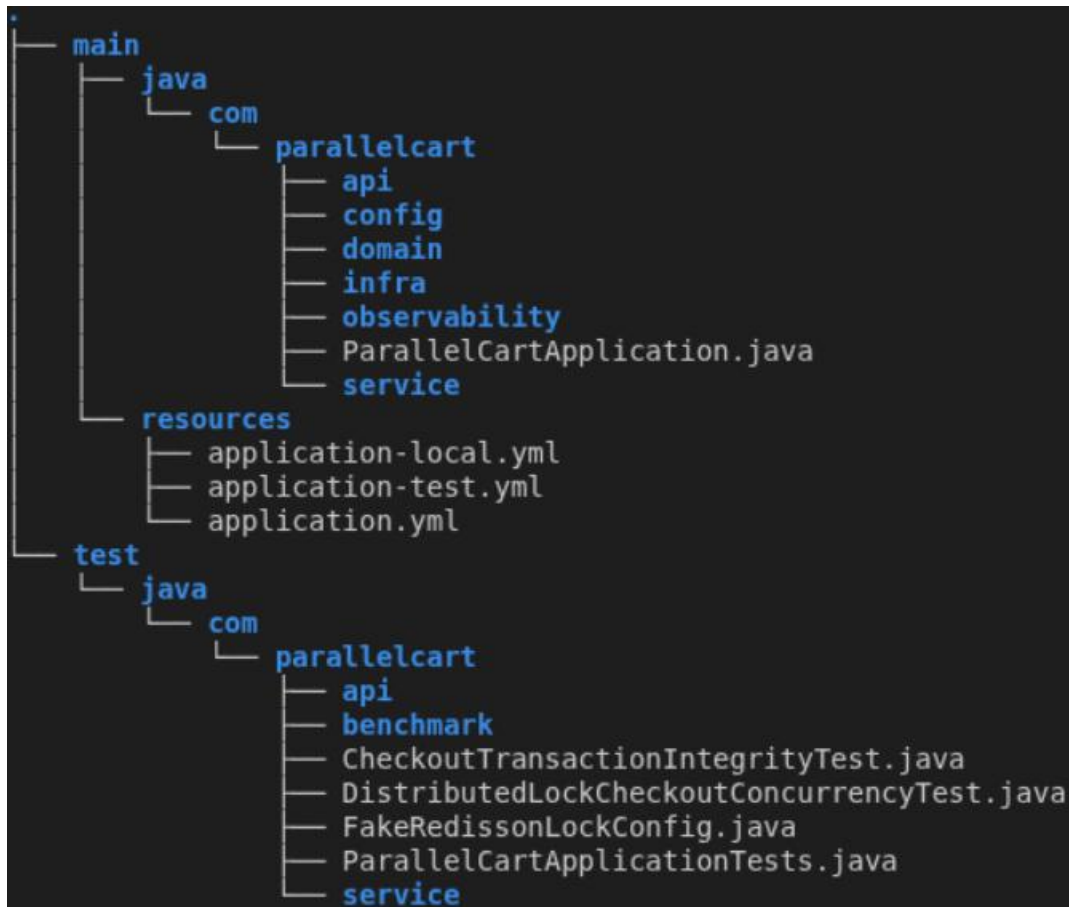
المكوّن	الدور في المشروع
Spring Boot	تطبيق الـ backend وتنظيم طبقات المشروع.
PostgreSQL	تخزين المستخدمين، المنتجات، السلالات، الطلبات، المدفوعات، المخزون، outbox، batch summaries.
Redis	Distributed cache و Distributed locks عبر Redisson.
Kafka	نشر أحداث order.created وتنفيذ الأعمال الخلفية.
Nginx	توزيع الطلبات على أكثر من instance باستخدام least_conn.

JMeter / k6	تنفيذ اختبارات الضغط وقياس زمن الاستجابة ومعدل الأخطاء.
Actuator / Micrometer	عرض metrics وتحليل bottlenecks أثناء التشغيل.

3. المعمارية العامة للنظام

تم تقسيم المشروع إلى طبقات واضحة حتى لا تختلط مسؤوليات الـ API مع منطق العمل أو التخزين أو القياس. هذا التقسيم جعل كل متطلب قابلاً للتتبع داخل الكود والاختبارات.

الحزمة	المسؤولية
api	Controllors و DTOs ومعالجة الأخطاء.
domain	Entities والنماذج الأساسية مثل Product و Inventory و Payment و Order و
service	واجهات الخدمات ومنطق العمل.
service.impl	التنفيذ الفعلي للـ checkout والكاش والـ batch والـ locks.
infra.repository	Spring Data JPA repositories.
infra.messaging	Outbox و Kafka publishers/consumers.
infra.batch	Scheduled job لجدد المبيعات اليومية.
config	Redis, Kafka, resource management, security, cache.
observability	AOP timing و Actuator bottleneck reports و metrics.

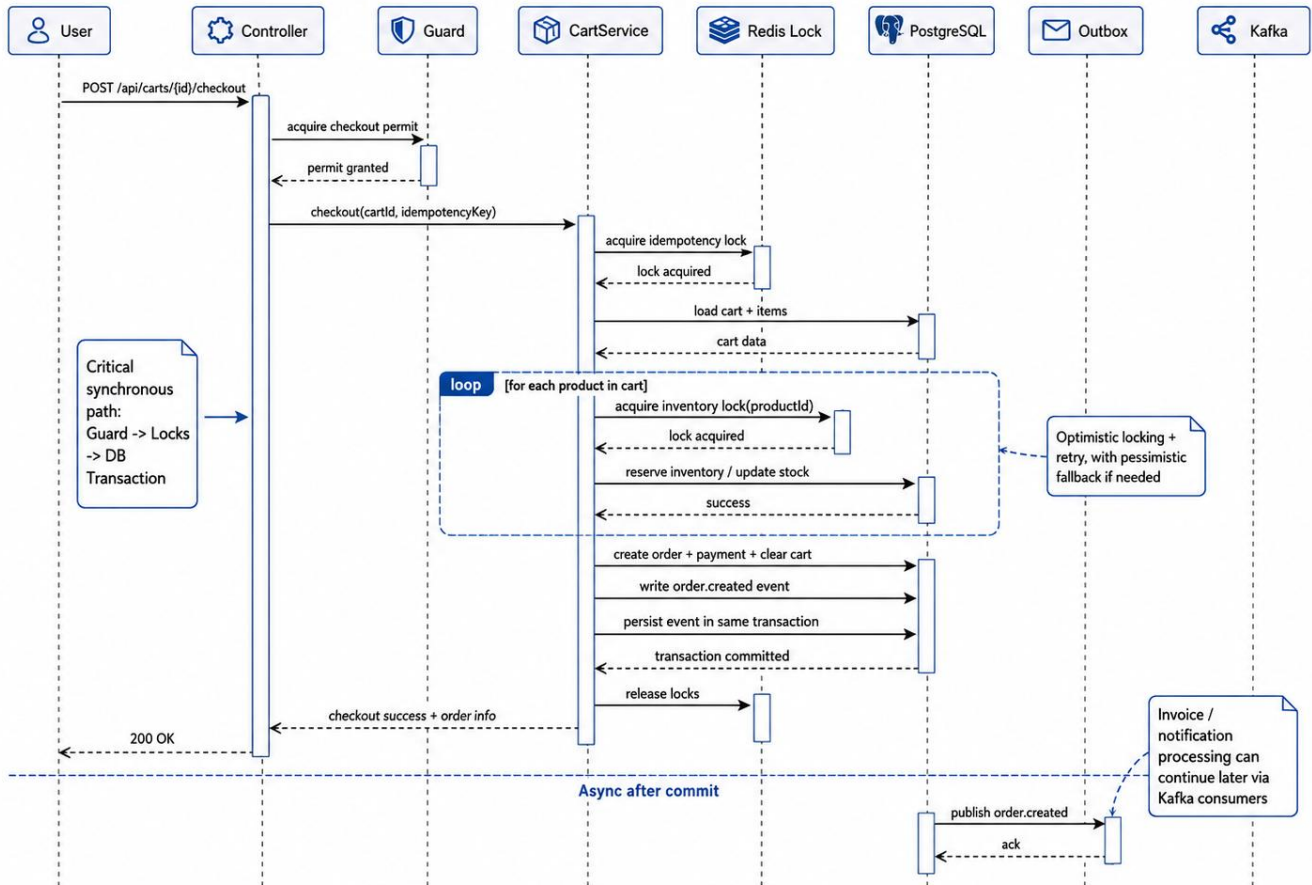


3.1 مسار checkout العام

1. يصل طلب checkout إلى CartController.
2. يمر الطلب أولاً على SemaphoreCheckoutSaturationGuard حتى لا يدخل عدد غير محدود من عمليات checkout في نفس الوقت.
3. يدخل الطلب إلى CartServiceImpl.checkout ضمن Transaction واحدة.
4. يتم أخذ قفل idempotency باستخدام Redisson حتى لا تتكرر نفس العملية بنفس المفتاح.
5. يتم تحميل السلة وعناصرها والتحقق أنها ليست فارغة.
6. لكل منتج في السلة، يتم أخذ قفل مخزون على مستوى productId.
7. يتم حجز المخزون باستخدام optimistic locking مع retry و jitter، ثم fallback إلى pessimistic lock عند الحاجة.
8. يتم إنشاء Order و Payment وحذف عناصر السلة وتسجيل OutboxEvent.
9. بعد نجاح commit يتم حذف cache entries المتأثرة.
10. يقوم OutboxPublisherJob لاحقاً بنشر order.created إلى Kafka، ثم تعمل consumers للفواتير والتنبيهات.

مخطط تسلسلي لمسار Checkout

Sequence Diagram - Checkout Flow



4. طريقة العمل على المتطلبات

تم التعامل مع المتطلبات بطريقة evidence-driven. أي أن كل مطلب لم يتم اعتباره منجزاً فقط بمجرد ذكره في التقرير، بل تم ربطه بألية في الكود، واختبار أو سكربت تشغيل، ونتيجة قابلة للعرض.

المرحلة	العمل المنجز
P0	تأسيس Spring Boot, Docker Compose, profiles, health/readiness.
P1	بناء نموذج المتجر الأساسي وواجهات المنتجات والسلة والcheckout.
P2	معالجة التزامن والمعاملات: optimistic/pessimistic locking, retry, rollback, idempotency.
P3	إضافة Kafka وOutbox والفواتير والتنبيهات وDLQ.
P4	إضافة Redis cache, invalidation, bounded executors, saturation guard.
P5	Batch processing للمبيعات اليومية و Nginx load balancing.
P6	AOP timing, stress tests, JMeter, bottleneck analysis, Actuator endpoints.

النتيجة أن المشروع يغطي المتطلبات العشرة على مستوى الكود والاختبار والقياس، مع ملاحظة أن بعض القياسات لا تعني دائماً تحسناً مباشراً في latency، بل قد تثبت تحسناً في جانب آخر مثل تقليل عدد استعلامات قاعدة البيانات أو إظهار مكان الاختناق بشكل أوضح.

5. تفصيل المتطلبات العشرة

5.1 حماية البيانات المشتركة من التضارب

معنى المتطلب: المطلوب منع حدوث race condition عندما يحاول أكثر من مستخدم تعديل نفس المورد، وأهم مورد في مشروعنا هو كمية المنتج في جدول المخزون.

ما تم تنفيذه:

- إضافة حقل version داخل Inventory باستخدام JPA @Version.
 - تنفيذ reserveInventoryWithRetry داخل CartServiceImpl.
 - إعادة المحاولة عند ObjectOptimisticLockingFailureException حتى ثلاث مرات.
 - استخدام jitter عشوائي بين المحاولات حتى لا تعود كل الخيوط للتصادم بنفس اللحظة.
 - استخدام findByProductForUpdate ك pessimistic fallback عند استمرار التعارض.
 - إضافة Redisson lock على مستوى productId لحماية نفس المورد عند وجود أكثر من app instance.
- التفسير الهندسي: تم اختيار optimistic locking كمسار أول لأنه لا يحجز الصفوف مسبقاً ويكون مناسباً عندما يكون التعارض محدوداً. وعند ارتفاع التنافس على منتج معين، يتحول النظام إلى pessimistic lock لضمان تسلسل التحديث على الصف. إضافة Redisson lock تجعل الحماية صالحة عندما يكون النظام موزعاً على app-1 و app-2، لأن القفل لا يبقى داخل JVM واحدة فقط.

الأدلة المقترحة للعرض: Inventory.java, InventoryRepository.java, CartServiceImpl.java, RedissonDistributedLockService.java, DistributedLockCheckoutConcurrencyTest, SQL validation بعد الضغط.

5.2 إدارة الموارد والتحكم بالطاقة الاستيعابية

معنى المتطلب: المطلوب ألا يسمح النظام بفتح عمليات متوازية غير محدودة تؤدي إلى استهلاك CPU أو الذاكرة أو connection pool بشكل مفرط.

ما تم تنفيذه:

- تكوين ThreadPoolTaskExecutor بحد أدنى وحد أعلى وحجم queue محدد.
 - استخدام CallerRunsPolicy بدل إسقاط العمل بصمت.
 - تكوين scheduler pool للمهام الخلفية.
 - استخدام SemaphoreCheckoutSaturationGuard لتحديد عدد عمليات checkout المتزامنة.
 - إرجاع خطأ منظم عند التشبع بدل انهيار التطبيق.
- التفسير الهندسي: تم اعتبار checkout أكثر عملية مكلفة وحساسة، لذلك تم وضع حد مباشر عليها. الهدف ليس جعل النظام يرفض الطلبات دائماً، بل حمايته من الانهيار عندما يتجاوز الضغط القدرة المتاحة. وجود queue محدد أفضل من queue لا نهاية لأنها لا تخفي المشكلة حتى تنفذ الذاكرة.
- الأدلة المقترحة للعرض: ResourceManagementConfig.java, SemaphoreCheckoutSaturationGuard.java, CartController.java, CartControllerSaturationTest, metrics الخاصة بال executor وال saturation.

5.3 المعالجة غير المتزامنة باستخدام Queues

معنى المتطلب: المطلوب نقل الأعمال التي لا يجب أن ينتظرها المستخدم خارج المسار الرئيسي للطلب، مثل الفواتير والتنبيهات.

ما تم تنفيذه:

- بعد نجاح إنشاء الطلب والدفع، يتم تسجيل OutboxEvent داخل نفس transaction.
 - OutboxPublisherJob • يقرأ الأحداث PENDING وينشرها إلى Kafka.
 - KafkaOrderEventPublisher • ينشر event باسم order.created.
 - InvoiceOrderCreatedConsumer • ينشئ invoice.
 - NotificationOrderCreatedConsumer • يسجل notification log.
 - KafkaConsumerErrorHandlingConfig • يضيف retry وDLQ في حال فشل consumer.
- التفسير الهندسي: لم يتم نشر Kafka event مباشرة من داخل checkout فقط، بل تم استخدام transactional outbox. السبب أن نشر event مباشرة قد يسبب dual-write inconsistency: ربما تنجح قاعدة البيانات ويفشل Kafka أو العكس. تسجيل outbox داخل نفس transaction يجعل الحدث جزءاً من الحقيقة المخزنة، ثم ينشر لاحقاً بطريقة قابلة للإعادة.
- الأدلة المقترحة للعرض: OutboxPublisherJob.java, OutboxEventService.java, InvoiceOrderCreatedConsumer.java, KafkaOrderEventPublisher.java, NotificationOrderCreatedConsumer.java, outbox SQL status.

```

paid_orders
-----
          100
(1 row)

payments
-----
          100
(1 row)

min_inventory
-----
        99994
(1 row)

negative_inventory
-----
              0
(1 row)

 status | count
-----+-----
PUBLISHED |    100
(1 row)

```

5.4 معالجة البيانات الضخمة على دفعات

معنى المتطلب: المطلوب وجود job خلفي يجرّد المبيعات اليومية على chunks بدل قراءة كل البيانات مرة واحدة في الذاكرة.

ما تم تنفيذه:

- DailySalesAggregationServiceImpl • يعالج الطلبات المدفوعة ضمن نافذة تاريخية ليوم معين.
- يتم تحميل orders باستخدام PageRequest وبحجم chunk قابل للضبط.
- DailySalesCheckpoint • يحتفظ بآخر order تمت معالجته وبعدد الطلبات والإيراد وعدد chunks.
- DailySalesSummary • يخزن النتيجة النهائية.
- DailySalesAggregationJob • يعمل بجدولة يومية على مبيعات اليوم السابق.
- max-chunks-per-run • يسمح بإيقاف المعالجة جزئياً ثم إكمالها لاحقاً.

التفسير الهندسي: القراءة على دفعات تمنع تحميل عدد كبير من الطلبات في الذاكرة وتسمح باستئناف العمل إذا توقف التطبيق أو تم تحديد عدد chunks في التشغيل الواحد. checkpoint يجعل ال batch job أقرب إلى تصميم production بدل تنفيذ مؤقت.

الأدلة المقترحة للعرض: `DailySalesAggregationServiceImpl.java`, `DailySalesAggregationJob.java`, `DailySalesCheckpoint.java`, `DailySalesAggregationCheckpointResumeTest.java`.

5.5 توزيع الأحمال

معنى المتطلب: المطلوب محاكاة توزيع الطلبات على أكثر من خادم وتوضيح سبب اختيار استراتيجية التوزيع.

ما تم تنفيذه:

- docker-compose يحتوي app-1 و app-2 ضمن profile باسم lb.
- Nginx يعمل أمام التطبيق على المنفذ 8088.
- nginx.conf يستخدم upstream باسم parallel_cart_app.
- تم استخدام least_conn لاختيار instance بعدد اتصالات أقل.
- تم إضافة header باسم X-Upstream-Addr لعرض السيرفر الذي استقبل الطلب.

التفسير الهندسي: استراتيجية least_conn أنسب من round-robin في هذا المشروع لأن زمن الطلبات ليس متساوياً. قراءة المنتجات قصيرة، بينما checkout أطول وقد ينتظر locks أو database. لذلك إرسال الطلب إلى instance ذات الاتصالات الأقل يقلل احتمال تحميل instance واحدة أكثر من غيرها.

الأدلة المقترحة للعرض: `docs/load-distribution-`, `docker/nginx/nginx.conf`, `docker-compose.yml`, `strategy.md`، صور curl مع X-Upstream-Addr.

5.6 التخزين المؤقت الموزع Redis Cache

معنى المتطلب: المطلوب تخزين المنتجات الأكثر طلباً في distributed cache لتقليل الاستعلامات المباشرة من قاعدة البيانات.

ما تم تنفيذه:

- CacheConfig يعرف RedisCacheManager مع caches للمنتجات والسلات والطلبات.
- ProductServiceImpl يستخدم Cacheable@ لـ listProducts و getProduct.
- CartServiceImpl يستخدم cache للسلة.
- OrderServiceImpl يستخدم cache للطلب وتاريخ طلبات المستخدم.
- CacheInvalidationServiceImpl يحذف مفاتيح الكاش بعد نجاح commit.
- تم قياس hits/misses و estimated DB queries avoided عبر instrumentation.

التفسير الهندسي: الكاش مفيد جداً للقراءات المتكررة، لكنه قد يصبح خطراً إذا لم يتم حذفه بعد تغيير البيانات. لذلك لم يتم حذف الكاش قبل نجاح transaction، بل بعد commit فقط. بهذه الطريقة لا يتم نشر حالة مبنية على معاملة فشلت لاحقاً. كما أن الكاش موزع عبر Redis، وبالتالي تستفيد منه app-1 و app-2 معاً.

الأدلة المقترحة للعرض: CartServiceImpl.java, ProductServiceImpl.java, CacheConfig.java, OrderServiceImpl.java, ProductServiceCachingTest, CacheInvalidationServiceImpl.java, Cache Analysis. جدول, OrderServiceCachingTest, CartServiceCachingTest

5.7 التحكم بالأقفال Distributed / Pessimistic / Optimistic

معنى المتطلب: المطلوب تطبيق optimistic locking أو pessimistic locking عند تحديث كميات المخزون الحساسة.

ما تم تنفيذه:

- Optimistic locking عبر Version@ في Inventory.
- Pessimistic locking عبر Lock(PESSIMISTIC_WRITE@ في InventoryRepository.
- Distributed locking عبر RedissonDistributedLockService.
- القفل الموزع للمخزون يستخدم key بالشكل id>.:inventory:product
- قفل idempotency يستخدم key بالشكل key>.:checkout:idempotency
- يتم تأجيل unlock إلى afterCompletion عندما تكون هناك transaction فعالة.

التفسير الهندسي: استخدام نوع واحد من الأقفال غير كافٍ في كل الحالات. optimistic lock سريع لكنه قد يعيد المحاولة كثيراً تحت الضغط. pessimistic lock آمن لكنه قد يقلل التوازي. distributed lock ضروري عندما يعمل أكثر من Instance. لذلك تم استخدام طبقات متعددة حسب مستوى الخطر.

الأدلة المقترحة للعرض: InventoryRepository.java, Inventory.java, RedissonDistributedLockService.java, CartServiceImpl.java, Contention Heatmap.

5.8 سلامة المعاملات ACID

معنى المتطلب: المطلوب أن تنجح عملية الدفع وتحديث المخزون وإنشاء الطلب كلها معاً أو تفشل كلها معاً، حتى مع وجود وصول متزامن.

ما تم تنفيذه:

- CartServiceImpl.checkout موسومة ب Transactional.@
- داخل نفس transaction يتم حجز المخزون، إنشاء order، إنشاء payment، حذف عناصر السلة، وتسجيل outbox event.
- عند أي خطأ، Spring يقوم بعمل rollback.
- تم اختبار rollback بحالة total غير صالح.
- تم اختبار idempotency بحيث نفس المفتاح يرجع نفس order ولا ينشئ duplicate order.

التفسير الهندسي: الهدف أن لا تظهر حالات جزئية مثل دفع بدون طلب أو طلب بدون خصم مخزون. وضع كل العمليات الحساسة ضمن transaction واحدة يجعل قاعدة البيانات هي الضامن النهائي للتكامل، بينما الأقفال وال idempotency تمنع تعارضات التنفيذ قبل الوصول إلى commit.

الأدلة المقترحة للعرض: CheckoutTransactionIntegrityTest, CartServiceImpl.java, DistributedLockCheckoutConcurrencyTest, SQL validation: paid_orders = payments, negative_inventory = 0.g

5.9 اختبار الاستقرار تحت الضغط

معنى المتطلب: المطلوب إثبات قدرة النظام على خدمة 100 مستخدم متزامن على الأقل دون انهيار أو فقدان بيانات.

ما تم تنفيذه:

- تم استخدام JMeter للاختبار read و checkout scenarios.
 - تم استخدام stress profiles بعدد Threads يصل إلى 100 في high-cache-enabled و hot-cache-enabled.
 - تم تشغيل checkout plan بحيث يحتوي POST add cart item ثم POST checkout.
 - تمت إضافة SQL validation بعد الاختبار للتحقق من paid orders و payments و المخزون و outbox.
 - تم إنشاء dashboards من JMeter reports للعرض.
- التفسير الهندسي: لم نعتمد فقط على أن HTTP requests رجعت 200. تم فحص قاعدة البيانات بعد الضغط للتأكد أن عدد المدفوعات يطابق عدد الطلبات، وأن المخزون لم يصبح سالباً، وأن outbox events أصبحت PUBLISHED. هذا أهم من مجرد زمن الاستجابة لأنه يثبت عدم فقدان البيانات.
- الأدلة المقترحة للعرض: JMeter dashboard, sql-validation.log, checkout.jtl, read.jtl, reports/bottleneck, SQL Validation. جدول.

5.10 القياس وتحديد الاختناقات

معنى المتطلب: المطلوب قياس زمن الاستجابة لأهم العمليات، تحديد bottleneck واحد على الأقل، وتقديم مقارنة رقمية قبل وبعد التحسين.

ما تم تنفيذه:

- PerformanceTimingAspect يسجل زمن product reads و checkout و inventory update.
 - BenchmarkMetricsService يسجل counters و timers و distribution summaries.
 - تمت إضافة actuator/bottlenecks/ و actuator/benchmarkreport/.
 - BottleneckAnalysisService يصف الوقت حسب buckets مثل Kafka/Outbox, DB queries, cache miss, lock contention, و connection wait.
 - تم تشغيل سيناريوهات hot-product و cache-enabled/cache-disabled لتحليل الاختناق.
- التفسير الهندسي: القياس لم يكن فقط average latency. تم تحليل p95 و p99, SQL validation, cache, hits/misses, lock wait, DB connection wait, و Kafka/outbox publish. من المهم التفريق بين bottleneck يؤثر على window كامل وبين ما يوقف HTTP checkout مباشرة. مثلاً Kafka/outbox ظهر كمساهم كبير في فترة الاختبار، لكن load و cart و inventory reservation هما الأهم داخل checkout المتزامن.
- الأدلة المقترحة للعرض: BottleneckAnalysisService.java, PerformanceTimingAspect.java, BottleneckAnalysisService.java, BenchmarkMetricsService.java, bottlenecks.json, JMeter metrics, reports/bottleneck.

6. عرض النتائج والقياسات

تم عرض النتائج على أساس هندسي، أي أن النتيجة لا تقاس فقط بزمن الاستجابة، بل يتم ربطها بصحة البيانات، معدل الأخطاء، عدد الاستعلامات المتجنبة، حالة outbox، وعدم وجود مخزون سالب. لذلك قُسمت النتائج إلى عدة مستويات: JMeter, SQL validation, Actuator bottlenecks, و cache/lock metrics.

6.1 منهجية الاختبار

نوع الدليل	ما يثبتته
Maven tests	سلامة منطق الخدمات وال caching وال rollback وال batch .resume
JMeter read plan	زمن استجابة قراءة المنتجات تحت عدد كبير من الطلبات.
JMeter checkout plan	قدرة checkout على العمل تحت ضغط مع add cart item ثم checkout.
SQL validation	مطابقة orders/payments وعدم وجود negative inventory وحالة outbox.
Actuator bottlenecks	توزيع الزمن بين DB, Kafka/outbox, cache, locks, resources.
Hot-product scenario	إظهار التنافس الحقيقي على منتج واحد بدل توزيع الضغط على منتجات كثيرة.

```
eto@teto-R0G-Strix-G614JU-G614JU:~/dev_code/parallel-cart$ ./jmeter/run-jmeter-before-after.sh both
+] down 3/3
✓ Volume parallel-cart_kafka_data Removed
✓ Volume parallel-cart_postgres_data Removed
✓ Volume parallel-cart_redis_data Removed
+] Building 3.0s (20/20) FINISHED
=> [internal] load local bake definitions
=> => reading from stdin 518B
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 576B
=> [internal] load metadata for docker.io/library/eclipse-temurin:21-jdk
=> [internal] load metadata for docker.io/library/eclipse-temurin:21-jre
=> [internal] load .dockerignore
=> => transferring context: 245B
=> [runtime 1/3] FROM docker.io/library/eclipse-temurin:21-jre@sha256:8ec353b20d3aab0758572236b81b9
=> [build 1/9] FROM docker.io/library/eclipse-temurin:21-jdk@sha256:762d16be48b976c97801d198e5085b1
=> [internal] load build context
=> => transferring context: 11.80kB
=> CACHED [runtime 2/3] WORKDIR /app
=> CACHED [build 2/9] WORKDIR /workspace
=> CACHED [build 3/9] COPY mvnw mvnw
=> CACHED [build 4/9] COPY .mvn .mvn
=> CACHED [build 5/9] COPY pom.xml pom.xml
```

6.2 نتائج JMeter الأساسية

يوضح الجدول التالي نتائج مختصرة من آخر تشغيل benchmark. الأرقام الأهم هنا هي أن جميع السيناريوهات انتهت بدون أخطاء، وأن high-cache-enabled و hot-cache-enabled عملا بعدد 100 thread.

السيناريو	الخطأ	العينات	الأخطاء	Avg ms	95p	99p
high-cache-enabled	read	4000	0	2.91	6	11
high-cache-enabled	checkout	200	0	25.80	37	111
hot-cache-enabled	read	4000	0	2.61	5	10
hot-cache-enabled	checkout	200	0	28.61	46	106
medium-cache-disabled	read	2400	0	3.08	6	11
medium-	read	2400	0	3.32	7	13

						cache-enabled
--	--	--	--	--	--	---------------

APDEX (Application Performance Index)

Apdex	T (Toleration threshold)	F (Frustration threshold)	Label
1.000	500 ms	1 sec 500 ms	Total
1.000	500 ms	1 sec 500 ms	GET /api/products/{id}
1.000	500 ms	1 sec 500 ms	GET /api/products

Requests Summary



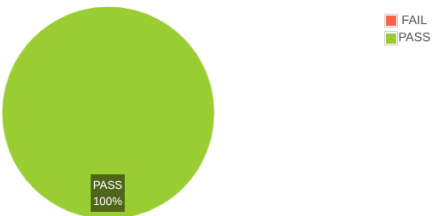
Statistics

Requests		Executions		Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	10000	0	0.00%	2.24	0	82	2.00	4.00	5.00	7.00	401.17	1301.80	73.80
GET /api/products	5000	0	0.00%	2.29	0	82	2.00	4.00	5.00	7.00	201.01	1209.02	36.71
GET /api/products/{id}	5000	0	0.00%	2.19	0	23	2.00	4.00	5.00	7.00	201.67	95.86	37.38

APDEX (Application Performance Index)

Apdex	T (Toleration threshold)	F (Frustration threshold)	Label
1.000	500 ms	1 sec 500 ms	Total
1.000	500 ms	1 sec 500 ms	POST checkout
1.000	500 ms	1 sec 500 ms	POST add cart item

Requests Summary



Statistics

Requests		Executions		Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	200	0	0.00%	24.83	8	114	23.00	31.90	36.00	113.88	10.29	4.32	2.71
POST add cart item	100	0	0.00%	23.52	8	114	21.50	27.90	28.95	114.00	5.15	2.28	1.22
POST checkout	100	0	0.00%	26.15	10	83	25.00	34.90	39.90	82.85	5.17	2.06	1.50

SQL Validation 6.3 بعد الضغط

بعد تشغيل الضغط تم فحص قاعدة البيانات. الهدف من هذا الفحص هو التأكد أن النظام لم يرجع نتائج HTTP صحيحة فقط، بل حافظ أيضاً على سلامة البيانات.

Outbox	Negative Inventory	Min Inventory	Payments	Paid Orders	السيناريو
PUBLISHED=20	0	99998	20	20	low-cache-enabled
PUBLISHED=60	0	99996	60	60	medium-cache-enabled
PUBLISHED=100	0	99994	100	100	high-cache-enabled
PUBLISHED=100	0	99930	100	100	hot-cache-enabled
PUBLISHED=60	0	99996	60	60	medium-cache-disabled

القراءة الهندسية لهذه النتائج أن عدد المدفوعات يساوي عدد الطلبات المدفوعة، ولا يوجد أي negative inventory، وجميع outbox events أصبحت PUBLISHED. هذا يثبت سلامة ACID وسلامة المعالجة غير المتزامنة بعد الضغط.

6.4 تحليل الكاش قبل وبعد

تم تشغيل سيناريو medium-cache-disabled بدون كاش، وسيناريوهات cache-enabled باستخدام Redis. من المهم كتابة النتيجة كما ظهرت بالأرقام: الكاش لم يثبت في هذا التشغيل القصير تحسناً واضحاً في end-to-end JMeter latency، لكن أثبت تقليل الضغط على قاعدة البيانات من خلال عدد cache hits والاستعلامات المتجنبة.

Estimated DB Queries Avoided	Hit Ratio	Product misses	Product hits	السيناريو
759	0.95	41	759	low-cache-enabled
2358	0.98	42	2358	medium-cache-enabled
3959	0.99	41	3959	high-cache-enabled
3958	0.99	42	3958	hot-cache-enabled
0	0.00	0	0	medium-cache-disabled

```

paid_orders
-----
          50
(1 row)

payments
-----
          50
(1 row)

min_inventory
-----
        99950
(1 row)

negative_inventory
-----
              0
(1 row)

 status | count
-----+-----
PUBLISHED |      50
(1 row)

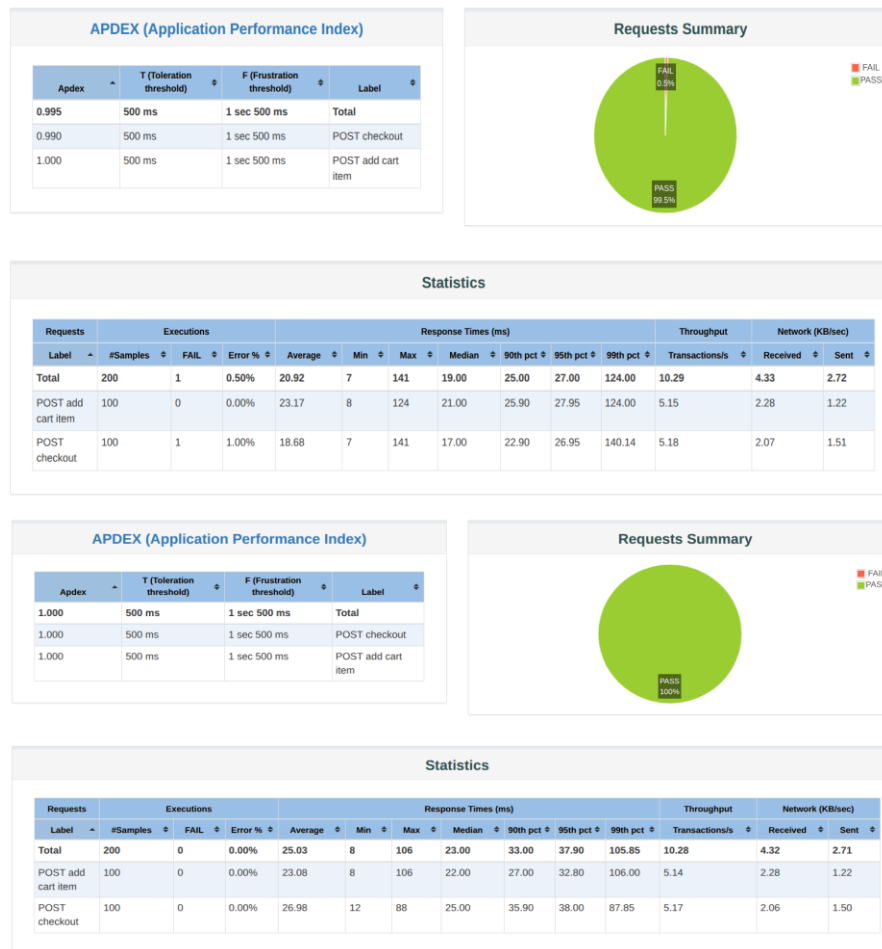
```

6.5 مقارنة before/after من JMeter History Git

تم أيضاً تشغيل مقارنة بين commit قبل التحسينات اللاحقة وبعدها. في هذه المقارنة انخفضت أخطاء checkout من 1/200 إلى 0/200، وتحسنت read latency، بينما checkout average latency لم يتحسن؛ لذلك تم عرض النتيجة بصدق هندسي بدون ادعاء تحسن غير موجود.

النسخة	السيناريو	Samples	Errors	Avg ms	p95 ms	p99 ms
Before	Read	10000	0	2.55	5.00	9.01
After	Read	10000	0	2.16	4.00	7.00
Before	Checkout	200	1	20.93	27.00	124.00
After	Checkout	200	0	25.03	36.10	91.15

في raw SQL للنسخة after ظهر paid_orders=100 و payments=100 و negative_inventory=0 في outbox. PUBLISHED=100 هذا يدل أن التحسين الأساسي كان في الثبات وسلامة النشر عبر outbox، وليس بالضرورة في متوسط زمن checkout.



APDEX (Application Performance Index)

Apdex	T (Toleration threshold)	F (Frustration threshold)	Label
1.000	500 ms	1 sec 500 ms	Total
1.000	500 ms	1 sec 500 ms	POST checkout
1.000	500 ms	1 sec 500 ms	POST add cart item

Requests Summary

Statistics

Requests Label	Executions			Response Times (ms)								Throughput		Network (KB/sec)	
	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent		
Total	200	0	0.00%	25.03	8	106	23.00	33.00	37.90	105.85	10.28	4.32	2.71		
POST add cart item	100	0	0.00%	23.08	8	106	22.00	27.00	32.80	106.00	5.14	2.28	1.22		
POST checkout	100	0	0.00%	26.98	12	88	25.00	35.90	38.00	87.85	5.17	2.06	1.50		

Bottleneck Ranking 6.6

أظهر تحليل الاختناقات أن Kafka/Outbox Publish كان أكبر مساهم ضمن نافذة benchmark، يليه Database Query Execution. لكن عند تحليل زمن HTTP checkout نفسه، كانت أكثر المراحل المتزامنة أهمية هي cart load و inventory reservation.

السيناريو	Rank 1	Rank 2	Rank 3
high-cache-enabled	Kafka/Outbox Publish %57.79	Database Query Execution 20.56%	Cache Invalidation %7.46
hot-cache-enabled	Kafka/Outbox Publish %54.92	Database Query Execution 22.02%	Inventory Lock Contention 6.12%
medium-cache-enabled	Kafka/Outbox Publish %43.98	Database Query Execution 26.37%	Cache Invalidation %7.84
medium-cache-disabled	Kafka/Outbox Publish %62.85	Database Query Execution 17.84%	Database Connection Wait 5.11%

النتيجة المهمة هنا أن bottleneck report لا يعطي رقماً واحداً فقط، بل يشرح أين يضع الوقت. مثلاً hot-product scenario يجعل product 1 نقطة التنافس الأساسية، وظهر Inventory Lock Contention ضمن أعلى خمسة اختناقات.

6.7 سيناريو Hot Product

تم تنفيذ سيناريو hot-cache-enabled بحيث يذهب 70% من المرور إلى product 1. الهدف من هذا السيناريو ليس تحسين latency، بل إثبات أن النظام يستطيع كشف التنافس على منتج معين وحماية المخزون تحته.

السيناريو	Top Product	Lock Count	Retry Count	Reservation Failures	Lock Wait Total ms
high-cache-enabled	2	6	0	0	49.72
hot-cache-enabled	1	70	0	0	183.88 / 147.78 حسب الملخص

ظهور product 1 كأعلى منتج في heatmap يعني أن القياس التقط مكان التنافس فعلاً. كما أن Reservation Failures بقيت 0، وهذا يدعم أن القفل وحجز المخزون يعالجان الضغط بدون فقدان بيانات.